

Governed & Scalable AI-assisted Analytics with dbt

Building AI acceleration you can actually trust



Jessica Stayton, Raini Laughlin

September 15-18, 2026

Meet the speakers



Jessica Stayton
(she/her)

Resident Architect



Raini Laughlin
(they/them)

Technical Instructor

Meet the TAs



Carol Ohms
(she/her)

Senior Resident Architect



Stephen Thibeault
(he/him)

Senior Resident Architect

Learning Objectives

- Describe where AI helps most in analytics engineering, and where it introduces risk
- Apply a review framework for AI-assisted changes (correctness, performance, maintainability, documentation)
- Identify governance decisions needed before scaling AI usage (who can access what context, what outputs are acceptable, how changes get approved)
- Establish team norms that prevent "AI drift" in definitions, naming, and documentation

What you'll leave with

You'll be able to:

- Spot where AI helps — and where it adds risk
- Set guardrails in dbt: AGENTS.md, skills, CI
- Make governance calls before you scale
- Prevent AI drift across a team

Scope & where to go deeper:

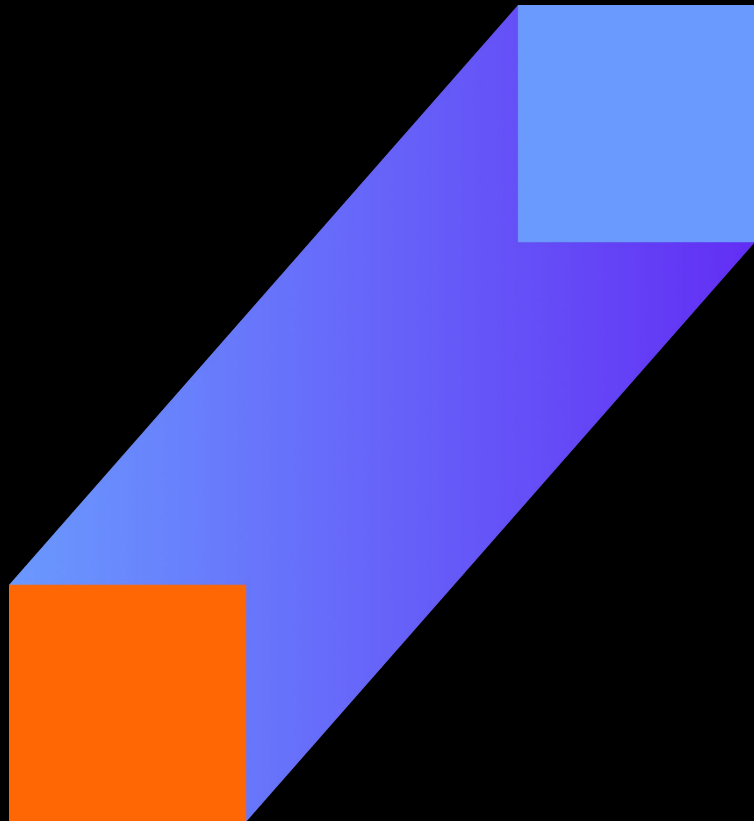
- We govern within dbt + Wizard
 - not the whole industry

Deeper elsewhere:

- *Creating context with dbt MCP Server* (Thu)
- *Accelerating analytics with AI* (Thu)
- Expert Lounge

How we'll work together

- **Sandbox** open the whole time
 - this is a build-along
- We'll **build one thing** together, and break a few on purpose
- **Ask as you go**
 - mistakes are the point
- Quick **survey** at the end shapes next year



Agenda

01

Meet the Project

- sandbox account
- patterns
- break it

02

AI Foundations

- reflection

03

Adding Governance to Development

- AGENTS.md
- fit it

04

Adding Governance to Production

- CI gates and rubric
- failed job

05

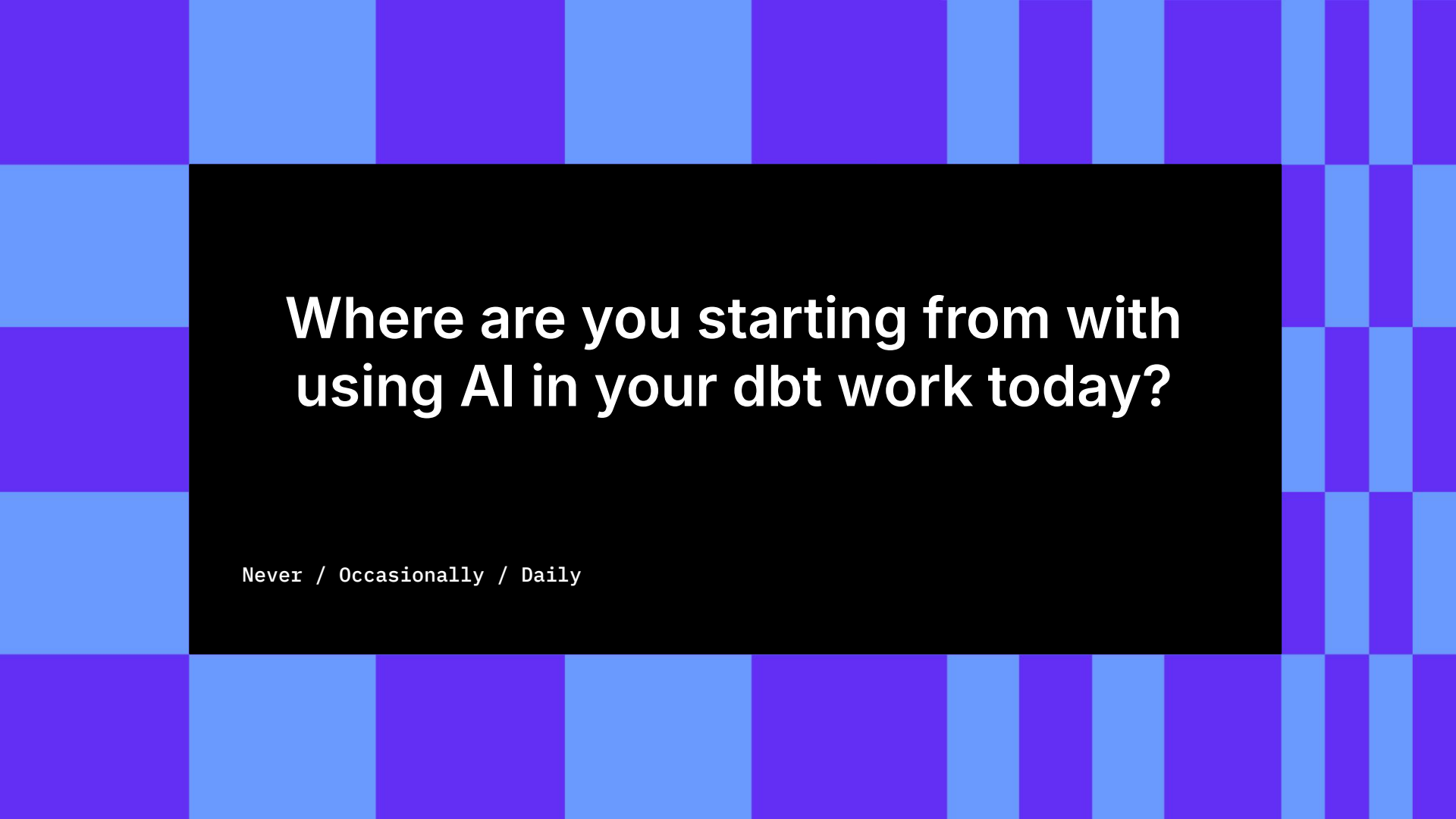
Scaling

- demo & resources

06

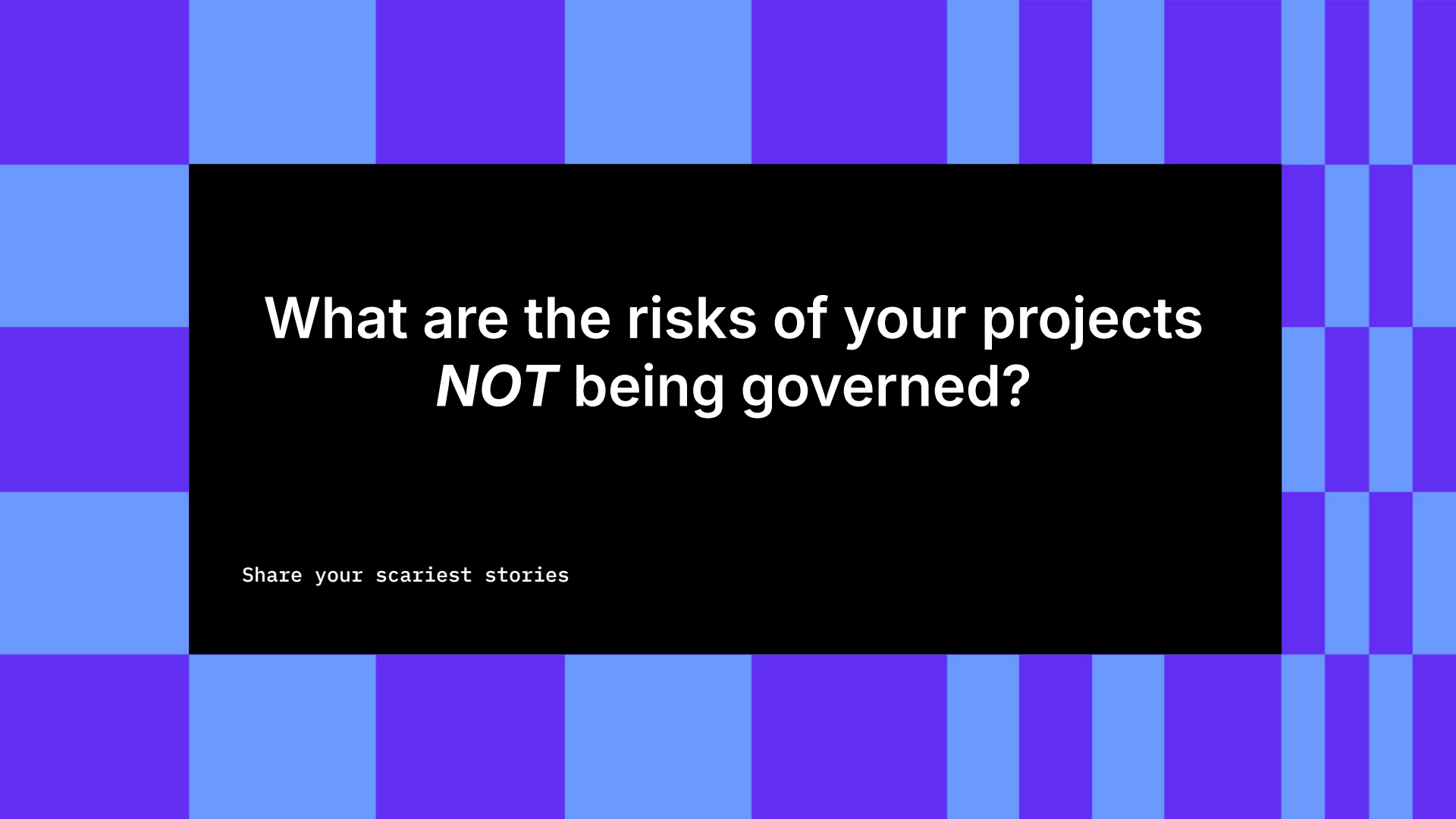
Wrap-Up and Survey

- training survey



Where are you starting from with using AI in your dbt work today?

Never / Occasionally / Daily



What are the risks of your projects *NOT* being governed?

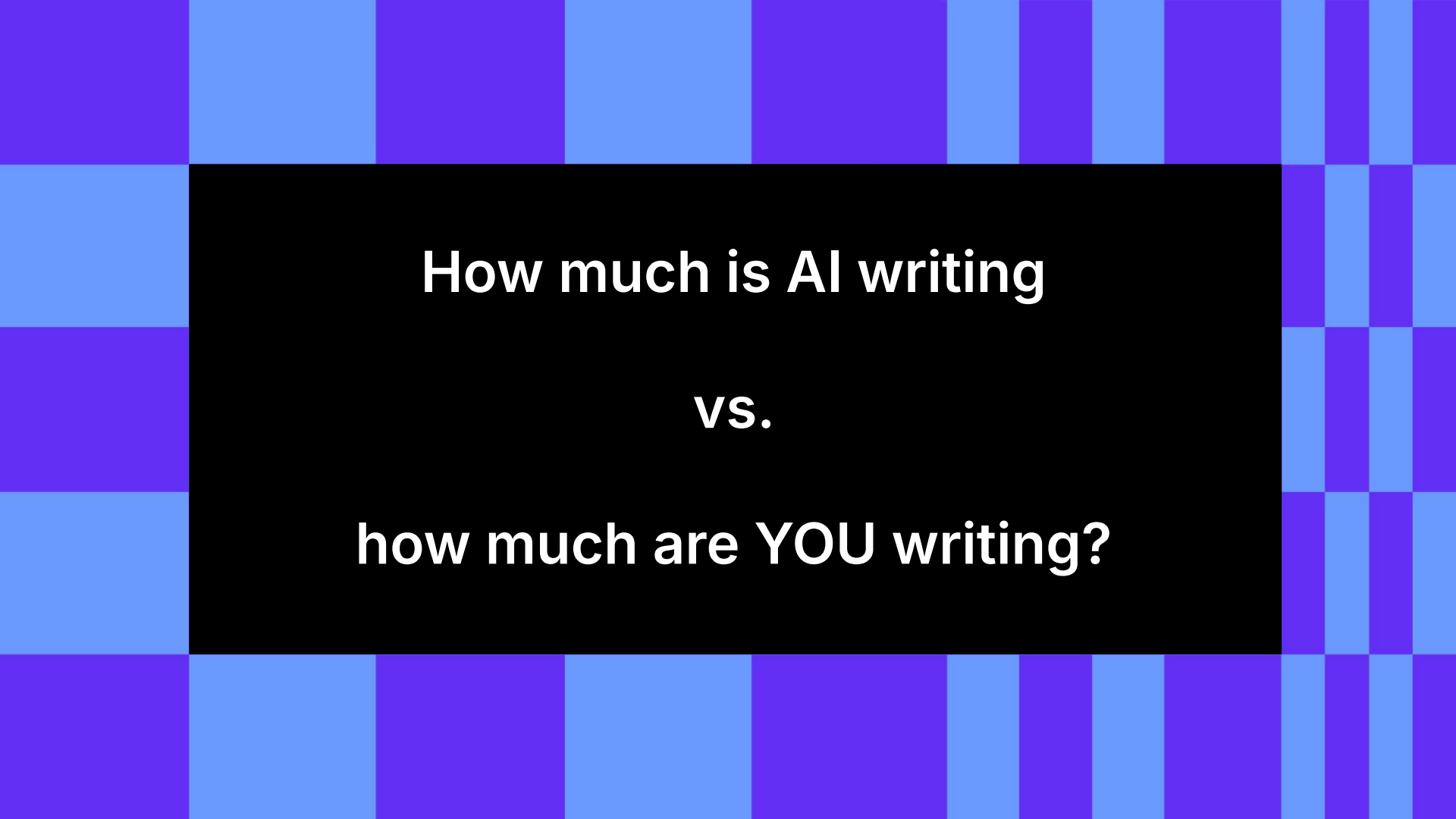
Share your scariest stories



What happens if you remove the guardrails?

Share your scariest stories





How much is AI writing

vs.

how much are YOU writing?





Meet the Project...

...then break it!

Sandbox Setup

Log out of any dbt accounts

- or use private/incognito session

Workshop Registration page:
workshops.us1.dbt.com/workshop

- Select **Getting Started with dbt**
- Enter the Passcode: **Summit2026!**
 - **Take note:** User ID and Password
 - Take a picture or screenshot



Workshop Signup

Please provide your first name, last name, and which workshop you will be attending. After submitting this form, you may log into the new account for your workshop.

First Name

Last Name

Workshop

Passcode

☐ I agree to dbt's [Terms of Service](#)

Meet the project, find the patterns

Built end-to-end: **abra** & **grimoire**

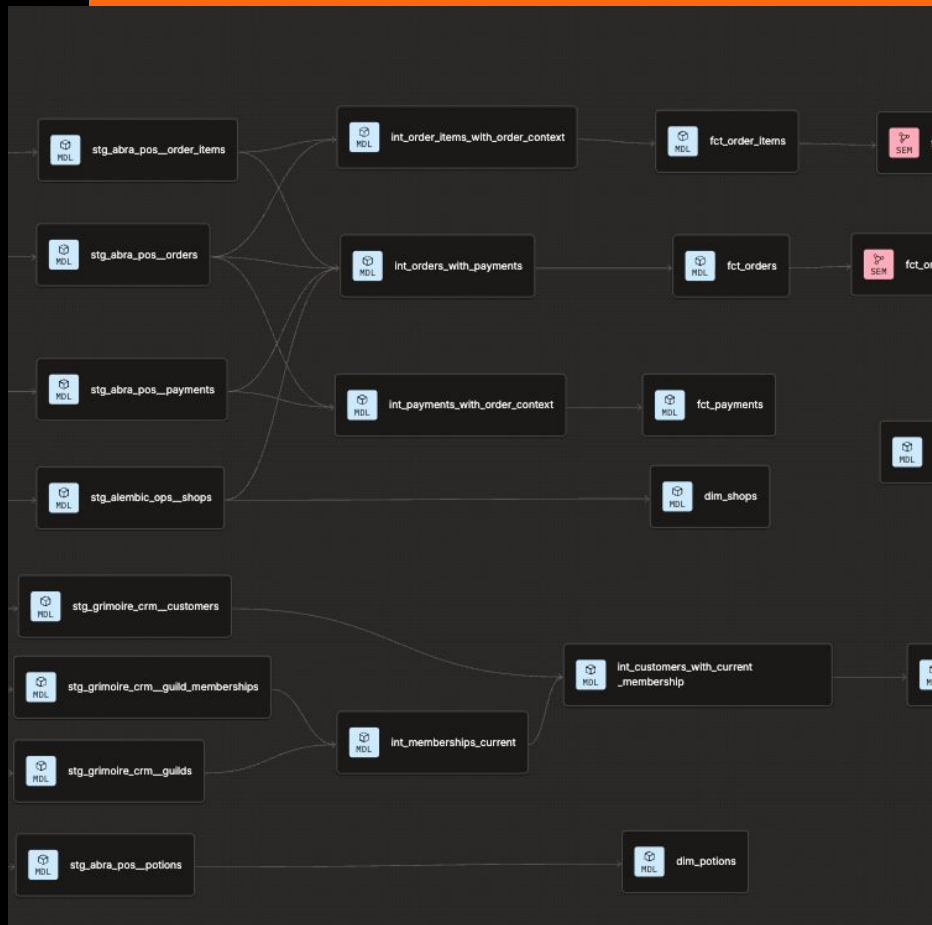
Missing: **alembic** (alembic_ops source)

Your job: hunt the patterns:

- naming, folders, tests, docs, grain

Write them in a scratch pad

- *you'll reuse these notes soon*



Now Break It!

Build alembic with no guardrails



Build models for the
alembic_ops source

- Watch what it does with the patterns you just wrote down
- Don't fix it, just observe

What happened?

Compare with your neighbor:

- Same prompt
- Turn to your neighbor
 - did you get the same output?
- Spoiler: it diverged

Name the risks:

- Where did it diverge?
- What breaks downstream?
- What would you catch by hand?



AI Foundations

Name what we just felt

The 80/20 framing

High Volume (80%)

Safe to Delegate to AI Tools

- Scaffolding and boilerplate
- First pass docs and test stubs
- First draft of semantic metrics
- Syntax
 - window functions
 - regex
 - date spines

High Stakes (20%)

Human in the Loop

- Business logic and edge cases
- Grain and join semantics
- Naming, layer, what ships
- Performance tradeoffs (i.e. materialization)
- **Governance**

Keep a human in the loop

The 20% is where you tell AI: check with me

- Delegate the draft → own the decision
- "Looks right" is the weakest bar
 - verify, don't trust
- **You** are accountable for what ships, not the agent

To keep the 20%, Own it!

The Design:

- How sources become trusted tables and metrics

The Success Criteria:

- What "correct" looks like
 - Not just a job ran without errors

The Rules:

- Business logic
- Grain
- Governance
- What "done" looks like

The Habits:

- Know the outcome
- Add on constraint per turn
- Align on meaning
- Keep changes small
- Prefer evidence over plausibility
- Reset when rounds add noise

Why AI misbehaves

AI Drift

- Same prompt, different answers each run
 - You saw this with your neighbor

Contributions

- Personal memory files
- User habits
- Software updates

AI Hallucinations

- Single instance of false confidence
 - e.g. a column that doesn't exist

Contributions:

- Gaps in training
- Guesswork instead of fact retrieval

Work the pattern

Treat AI as a collaborator, not a vending machine

Explore:

- Map the source
- Flag the casts/renames
- List assumptions *before* any SQL

Implement:

- Build in small and reviewable chunks

Plan:

- Propose:
 - Columns
 - Grain
 - Schema.yml
- Human checkpoint for "done"

Verify:

- compile/build
- Check grain and row counts
- Add a test that AI couldn't write

Guardrails

Three layers:

Guide

- context files (AGENTS.md, skills)

Enforce

- CI/CD gates on every PR

Runtime

- deployment, semantic layer, mesh

Two ways AI meets dbt:

- AI that builds your dbt project
- dbt as the trusted context AI consumes

Same prompt, now governed

Same prompt as before

- now with guardrails in place

Convention-correct

- naming, tests, documented grain

Fewer stops

- fewer corrections, output you can trust

This is the difference **context** makes



Adding governance to development

institutionalizing knowledge to prevent drift

Spoiler Alert, you started this

Remember the patterns you jotted down in Section 2?

- Rename that scratch pad → AGENTS.md
- Formalize:
 - layers
 - naming conventions
 - grain rules
 - where to look
- Your notes were governance all along!

"AI-ready"

Team Readiness

Level 1

Humans clear before handing off

👍 Test: can you explain the project to a new hire in writing, no meeting?

- Data stack, data, procedures, conventions, basic CI

Project Readiness

Level 2

Trustworthy enough for autonomy



Agentic end-to-end work

- tests/contracts, semantic layer, clean lineage, compiles everywhere

L1 without L2 is fine, never L2 without L1

The files that guide AI

The files

AGENTS.md

- what the agent should know/do
 - context, layers, naming, grain

SECURITY.md

- what the agent must **not** do:
 - no prod writes, no creds, stop-and-ask

skills/SKILLS.md

- reusable, load-when-relevant instructions
 - not in dbt Studio yet — supported locally

How to leverage them

- One shared source of truth
→ less drift
- Wizard reads AGENTS.md
+ skills by default

Build a skill from a pattern

A skill for writing skills

Point it at the staging pattern you already found

Output:

- a reusable "build a staging model" skill

Capture the pattern once → anyone reproduces it

You built the guardrails, now watch them work

Build the intermediate + marts skills the same way

Point your skills at the alembic source

- generate the missing slice

This has been the goal all along!

- watch your guardrails work!



Revise & re-run

You're teaching AI when to check in

- Feed it an under-documented source
 - what happens?
- Good skill:
 - stops and asks instead of guessing
- Add prompt-backs:
 - "if context is missing, ask before building"

Govern the rules like code

The question:

*Who can edit the rules vs.
propose a change, and on what?*

Shared assets

- AGENTS.md
- SECURITY.md
- skills

The answer:

*Review config the same way
you review code*

- CODEOWNERS → owner notified on every change
- Guardrails are only trustworthy if they're governed too



Adding governance to prod

Explicit boundaries for risk and action

Guardrails in action

Building with AI is one thing, shipping what it built is another

- Now:
 - review
 - promotion
 - production
- The stakes rise the closer you get to prod
- Same discipline, higher-consequence surface

Review as a discipline + the rubric

The rubric:

Correctness

- business logic
- null handling
- time logic

Tests

- performance
- maintainability

Governance

- operations

Review this PR:

- AI-generated, looks clean, passes preview
- Where does it fail the rubric?
- "Looks right" is the weakest bar there is

CI gates & agentic PR review

CI checks business logic, not just formatting

- An agent can review the PR
 - a human still signs off
- Wizard auto-tags its sessions → audit trail for free

Human-in-the-loop progression

Three-step progression:

Ask vs. auto

- start in ask mode
- earn auto

Prompt-backs

- make AI stop when context is missing

Sign-off

- a human approves before prod is touched

You decide where judgment is non-negotiable

RBAC, Approval Gate & Audit Trail

Read vs. Act

Who can do what:

- Read-only diagnosis ≠ permission to act
- Grant action deliberately, per role

The Paper Trail

prove it later:

- Approval gates on high-consequence actions
- Audit trail = who did what, when
- Governance = answerable, not just careful

Debug a failed job with Wizard

A deploy job failed, now what?

- Click Debug with Wizard → diagnosis
- You decide the fix
 - *the agent doesn't act alone*
- Compare with your neighbor
 - same diagnosis?



Scaling: consistency is an infrastructure problem

One person's pattern becomes shared infrastructure

Consistency across teams

You built a skill that works... for YOU

- Now: how does it work for the whole team?
- The problem isn't skill, it's consistency
- 40 developers, 40 private rule sets = drift at scale

Scale within a project

Make it Shared

shared assets:

- Skills + AGENTS.md live in the repo
 - everyone gets them
- One shared PR-review agent
 - same rubric for all

Make it Enforced

keep it owned:

- CODEOWNERS notifies owners when rules change
- Propose via PR, not silent edits
- Same discipline as feature branches

Across projects & beyond dbt

Coming Soon

many projects / Mesh:

- Sharing skills across projects
→ native support coming
- Watch for announcements
 - don't build workarounds yet

Beyond dbt

outside dbt:

- MCP, Runlayer, plugins = the next frontier
 - see Creating context with dbt MCP Server session on Thurs
- On-demand course coming this fall



Wrap-up and next steps

We did it!

Recap the arc

- **Explore** → felt the risk
- **Named it:** 80/20, drift, the AI workflow
- **Fixed the inputs:** AGENTS.md + skills
- **Governed the outputs:** review, sign-off, audit trail
- **Scaled the pattern,** not the chaos

LLMs reduce typing, not responsibility

Training Survey

*Governed & Scalable AI-assisted
Analytics with dbt*

Please complete the survey

- Scan QR
- link in the README

Questions?



Thank you!

Jessica, Raini, Carol & Stephen

